Encaminhamento Distribuído Baseado em WebRTC para Streaming Escalável de Vídeo Volumétrico

Matthias De Fré · Casper Haems · Jeroen van der Hooft · Tim Wauters · Filip De Turck
IDLab, Ghent University – imec, Gante, Bélgica

NOSSDAV '26 (Workshop on Network and Operating System Support for Digital Audio and Video), 4–8 de abril de 2026, Hong Kong. DOI: 10.1145/3798065.3798071. Licença original: CC BY 4.0.

Tradução não oficial para o português (pt-BR) de *Distributed WebRTC-Based Forwarding for Scalable Volumetric Video Streaming*. Termos técnicos consagrados (como *bitrate*, *throughput*, *jitter*) foram mantidos em inglês quando soam mais naturais assim. Figuras e imagens foram omitidas (permanecem apenas as legendas traduzidas); tabelas e equações foram reproduzidas. A lista de referências é mantida no idioma original. Em caso de dúvida, vale o texto original.

RESUMO

À medida que a mídia imersiva se torna mais acessível, surgiram versões virtuais de experiências do mundo real, como shows e conferências, viabilizadas por pipelines de streaming volumétrico para realidade virtual (VR). A representação do usuário é central nesses sistemas, e as nuvens de pontos são amplamente adotadas para avatares realistas por equilibrarem qualidade e desempenho. No entanto, a maioria dos sistemas tem dificuldade para escalar para um número maior de usuários. Embora trabalhos recentes tenham proposto arquiteturas mais escaláveis, elas costumam se concentrar em otimizações computacionais e não escalam para muitos usuários por causa de gargalos de rede. Para preencher essa lacuna, apresentamos um pipeline de streaming volumétrico open source, modular e distribuído, baseado em WebRTC, que emprega múltiplas unidades de encaminhamento seletivo (SFUs) para melhorar latência, throughput e qualidade em comparação com uma SFU centralizada. Os resultados mostram que, com 64 usuários, a configuração distribuída recebe 147% mais pontos com latência de transporte comparável e reduz a latência de pico com menos usuários. Além disso, o uso de adaptação de qualidade permite latência estável em diferentes escalas, de aproximadamente 25 ms.

Palavras-chave: Vídeo volumétrico, conferência, realidade virtual, WebRTC

1 INTRODUÇÃO

Nos últimos anos, a mídia imersiva se expandiu tanto no setor de entretenimento quanto no profissional, com melhorias no desempenho das redes e na entrega de conteúdo [20]. A disponibilidade de headsets de realidade virtual (VR) leves (HMDs, *head-mounted displays*) acelerou ainda mais a adoção, posicionando a mídia imersiva como meio para educação, colaboração remota e conferências virtuais [32, 19]. Em sistemas imersivos, a representação do usuário varia conforme os requisitos da aplicação: casos de uso colaborativos costumam empregar avatares sintéticos de baixa fidelidade para dar suporte à interação em tempo real [37], enquanto conferências virtuais priorizam o realismo visual [35]. Em geral, esses cenários permanecem limitados em escalabilidade, normalmente suportando apenas um número restrito de participantes. Trabalhos recentes trataram essas limitações com arquiteturas de rede e pipelines de renderização mais eficientes [9]. Esses avanços viabilizam ambientes virtuais complexos em larga escala, embora exijam mecanismos sofisticados para equilibrar escalabilidade, latência e qualidade entre os clientes [13]. A escolha do protocolo de transporte afeta significativamente o desempenho e a escalabilidade de sistemas imersivos. Soluções baseadas em TCP, como o DASH de baixa latência (LL-DASH), oferecem entrega confiável e qualidade adaptativa, mas sofrem com alta latência e interatividade limitada [33]. Em contraste, protocolos baseados em UDP, incluindo o WebRTC, buscam a menor latência possível ao custo da falta de entrega confiável. Arquiteturas escaláveis construídas sobre esses protocolos aumentam ainda mais o número de usuários simultâneos [2, 27], embora alcançar uma implantação verdadeiramente em escala global continue desafiador. Para enfrentar essas deficiências, este artigo propõe um novo pipeline de videoconferência volumétrica muitos-para-muitos que usa unidades de encaminhamento seletivo (SFUs) WebRTC distribuídas para aumentar a escalabilidade e o desempenho geral, com as seguintes contribuições:

- Um pipeline de streaming volumétrico muitos-para-muitos baseado em WebRTC e open source [25], que usa múltiplas SFUs para streaming distribuído eficiente em largura de banda;
- Uma avaliação em larga escala que analisa a escalabilidade para diferentes números de SFUs interconectadas e de clientes.

2 FUNDAMENTAÇÃO E TRABALHOS RELACIONADOS

Esta seção revisa trabalhos relacionados sobre entrega escalável de vídeo volumétrico, com foco nas escolhas de projeto que afetam a qualidade e a transmissão em tempo real, e sobre arquiteturas escaláveis.

Entrega de conteúdo volumétrico. Sistemas imersivos em tempo real costumam adotar representações em nuvem de pontos por sua simplicidade e eficiência [8]. Embora as nuvens de pontos sejam adequadas para conferências virtuais, suas altas taxas de bits dificultam a transmissão em tempo real [14]. Certos codecs alcançam taxas de bits comparáveis às de vídeo, mas com alta latência [21, 29]. Em contraste, codecs de tempo real como o Draco [12] trocam maior largura de banda por baixa latência. Além disso, a largura de banda pode ser reduzida ainda mais com técnicas de adaptação de qualidade, como *tiling* (ladrilhamento) [30] e codificação escalável [10], que permitem ajuste dinâmico com base no campo de visão (FoV) do usuário e nas condições da rede, reduzindo as taxas de bits e preservando a qualidade visual [17]. Por essas razões, usaremos nuvens de pontos codificadas com Draco para garantir desempenho escalável e em tempo real.

Figura 1: Arquitetura de SFUs distribuídas proposta para streaming volumétrico muitos-para-muitos, com suporte a encaminhamento de conteúdo entre múltiplas SFUs. Cada cliente pode atuar como remetente, receptor ou ambos.
(imagem omitida nesta tradução; consulte o artigo original)

Streaming de vídeo de baixa latência. Vários sistemas imersivos adaptaram soluções de transporte tradicionais para dar suporte a conteúdo volumétrico [3]. Para cenários que exigem alta confiabilidade, costumam ser escolhidos protocolos baseados em TCP, como o LL-DASH. No entanto, mesmo essa variante de baixa latência ainda apresenta latências altas, limitando a interatividade [4]. Em contraste, soluções baseadas em UDP, como o WebRTC, priorizam a baixa latência em detrimento de uma confiabilidade estrita, embora esses recursos possam ser implementados na camada de aplicação [24]. Embora os métodos baseados em UDP não ofereçam estimativa de largura de banda de forma inerente, algoritmos como o Google Congestion Control (GCC) [6] são frequentemente empregados para fornecer essa funcionalidade. Com base na largura de banda estimada e, potencialmente, no FoV do usuário, algoritmos adaptativos determinam então a qualidade mais adequada para cada usuário [34]. Abordagens mais avançadas refinam ainda mais esse processo intercalando tipos de quadros ou combinando múltiplos níveis de qualidade para otimizar a qualidade visual [5, 36]. Por essas razões, utilizaremos o WebRTC juntamente com um estimador de largura de banda para viabilizar streaming adaptativo de baixa latência.

Arquiteturas de streaming escaláveis. Cenários com muitos usuários frequentemente empregam servidores de borda (*edge*) localizados mais perto dos clientes para aumentar a escalabilidade [22, 26]. Alguns sistemas distinguem conexões de remetente e de receptor. Nessas arquiteturas, um cliente envia seu conteúdo para um único servidor, mas recebe conteúdo de vários servidores [18]. Esse projeto ajuda a lidar com o desequilíbrio entre a largura de banda de upload limitada do usuário e a capacidade de download comparativamente maior. No entanto, se muitos clientes solicitam o mesmo conteúdo a um único servidor, esse servidor se torna um gargalo [7]. Além disso, quando vários clientes próximos solicitam conteúdo idêntico a um servidor distante, os mesmos dados precisam percorrer longos caminhos de rede, aumentando a contenção nos enlaces compartilhados e os atrasos de fila [38]. Outros sistemas distribuídos restringem cada usuário a uma única conexão bidirecional, com o servidor conectado sendo responsável por encaminhar o conteúdo aos demais servidores. Tais sistemas introduzem latência adicional pelo fato de cada servidor ter de processar cada pacote [11]. Contudo, como os clientes estão distribuídos entre todos os servidores, a carga computacional total também é reduzida. Quando vários clientes conectados ao mesmo servidor solicitam conteúdo idêntico, arquiteturas baseadas em encaminhamento são vantajosas, pois o conteúdo pode ser retransmitido por enlaces de alta capacidade, com apenas o último salto até os clientes dependendo de conexões de consumidor [1]. No entanto, os sistemas existentes [18] de ambos os tipos não oferecem suporte a conteúdo volumétrico. Para aumentar a escalabilidade e reduzir o desperdício de largura de banda em enlaces de rede compartilhados, será usada uma abordagem de encaminhamento.

3 ARQUITETURA DO SISTEMA

Esta seção apresenta o pipeline proposto, escalável e distribuído, baseado em SFUs e WebRTC (Figura 1), descrevendo o fluxo de streaming, o gerenciamento de sessões, os mecanismos WebRTC de baixa latência, a troca de mensagens do protocolo de descrição de sessão (SDP) em um cenário distribuído e as principais escolhas de projeto da SFU.

Pipeline de vídeo volumétrico. Primeiro, o conteúdo é capturado com uma câmera de profundidade para obter quadros de cor e profundidade e convertê-los em uma nuvem de pontos. A nuvem de pontos resultante é então particionada em múltiplas descrições, para permitir a adaptação de qualidade e a codificação em paralelo. Por fim, a nuvem de pontos codificada é enviada à SFU WebRTC. No receptor, as descrições são entregues pela SFU, decodificadas em paralelo, mescladas em uma única nuvem de pontos e exibidas em um HMD.

Gerenciador de sessões. O gerenciador de sessões é responsável pela coordenação das sessões e pelo provisionamento das SFUs. Antes de transmitir conteúdo, os clientes registram suas *tracks* e parâmetros, garantindo consistência entre clientes e SFUs. Em seguida, o gerenciador de sessões instrui a SFU de destino a alocar recursos e, quando ela estiver pronta, devolve os detalhes da conexão ao cliente.

Entrega escalável em tempo real com WebRTC. Sistemas baseados em unidades de controle multiponto (MCU) agregam os fluxos em uma única saída composta, oferecendo melhor sincronização e menor uso de largura de banda ao custo de maior latência. Em contraste, sistemas baseados em SFU encaminham os pacotes diretamente aos clientes inscritos, permitindo menor latência e melhor escalabilidade. Embora o WebRTC não ofereça entrega confiável de forma inerente, retransmissões baseadas em reconhecimento negativo (NACK) são comumente usadas para reduzir o número de quadros não decodificáveis. Diferentemente da confiabilidade do TCP, essa abordagem permite que as aplicações troquem confiabilidade por latência, reduzindo o número de solicitações de retransmissão. Para evitar a perda de pacotes, sistemas baseados em WebRTC empregam algoritmos de controle de congestionamento, como o GCC, que viabilizam baixa latência sob condições de rede variáveis. Esses controladores estimam a largura de banda disponível e o atraso, permitindo que as aplicações adaptem o conteúdo antes que a rede se degrade.

Figura 2: Setup bare-metal usado nos experimentos. Cada nó SFU pode atuar como uma SFU normal ou como um simples nó comutador, e cada nó cliente pode hospedar uma ou mais aplicações cliente WebRTC.
(imagem omitida nesta tradução; consulte o artigo original)

Troca de mensagens SDP. No WebRTC, as conexões são estabelecidas por meio da troca de mensagens SDP que negociam os codecs suportados, os parâmetros de transporte e os candidatos de rede [23]. A negociação usa uma máquina de estados que impede a renegociação até que a troca atual seja concluída. Embora esse processo seja direto em arquiteturas com uma única SFU, estendê-lo a múltiplas SFUs introduz desafios adicionais de coordenação. Quando um novo cliente se conecta a uma SFU, todas as demais SFUs precisam ser notificadas, por renegociação SDP, de que há novas *tracks* WebRTC disponíveis. Tentativas concorrentes de renegociação podem levar a máquina de estados

da negociação a estados inválidos. Para evitar esses conflitos, é necessária uma ordenação determinística entre as SFUs, por exemplo usando um ID, para decidir qual nó inicia a troca SDP e qual responde.

Projeto da SFU distribuída. Por padrão, clientes conectados a uma SFU não conseguem receber conteúdo de clientes conectados a outra. Para permitir a comunicação entre SFUs, todas as SFUs de uma sessão estabelecem interconexões e encaminham os pacotes de áudio e vídeo relevantes dos clientes para as SFUs interessadas. Esse processo é transparente para os clientes, que mantêm uma única conexão com uma SFU, preservando a compatibilidade com as implementações WebRTC existentes. A SFU é agnóstica ao conteúdo e suporta a transmissão de mídia volumétrica codificada com codecs arbitrários. Embora a SFU permaneça independente do conteúdo, um módulo de adaptação de qualidade ajusta o conteúdo transmitido com base nas estimativas de largura de banda, indo da seleção de *tracks* em regime de melhor esforço (*best-effort*) até abordagens baseadas em codificação por múltiplas descrições (MDC) que equilibram a qualidade entre os clientes visíveis.

4 IMPLEMENTAÇÃO E AVALIAÇÃO

Esta seção descreve a implementação do pipeline distribuído, os cenários de avaliação e os métodos de adaptação de qualidade, as métricas consideradas e os resultados.

4.1 Detalhes de implementação

O pipeline é composto por vários módulos. No centro está a interface gráfica, implementada em Unity [31], que renderiza o conteúdo volumétrico e o exibe em VR. A aplicação se comunica com um módulo de câmera de profundidade que gera nuvens de pontos, posteriormente codificadas por um codificador baseado em MDC e comprimidas com Draco. Os componentes de captura e codificação são implementados em C++, enquanto o peer WebRTC, a SFU e o gerenciador de sessões são implementados em Golang, com integração ao Unity fornecida por meio de uma biblioteca C++. O pipeline suporta os modos *headful* e *headless*. No modo *headful*, os dados são capturados ao vivo e renderizados em VR, enquanto o modo *headless* desativa a captura e a renderização e pode usar conteúdo pré-codificado para focar o desempenho de rede e de adaptação de qualidade. Como esta avaliação se concentra em comparar arquiteturas de rede de SFUs WebRTC centralizadas e distribuídas, a maioria dos experimentos usa o modo *headless* com dados pré-codificados, e uma avaliação completa, *headful* e de ponta a ponta, é apresentada na Seção 4.7. O GCC foi selecionado como algoritmo de controle de congestionamento, com a taxa de bits mínima configurada em 15 Mbit/s. Com base nas estimativas de largura de banda fornecidas pelo GCC, a qualidade pode ser adaptada por diferentes métodos. A estratégia específica de adaptação depende dos cenários de avaliação descritos na Seção 4.3.

4.2 Configuração experimental

Os experimentos são executados no Virtual Wall [15], um testbed de nós bare-metal interconectados. Cada nó executa Ubuntu 22.04 e tem as seguintes especificações: *CPU: Intel i5-9500 (3,0 GHz), RAM: 64 GB DDR4 (3200 MHz), 5 NICs: 1 Gbit/s.* A Figura 2 mostra a topologia física da rede. A topologia compreende quatro sites que emulam localizações distintas, interconectados por um nó central. Enlaces diretos adicionais conectam o Site 3 ao Site 1 e o Site 4 ao Site 2, fornecendo caminhos de roteamento alternativos que contornam o nó compartilhado. A latência entre os sites é emulada por meio de controle de tráfego (*traffic control*). Os nós de cliente hospedam uma ou mais instâncias de cliente *headless*, enquanto os nós de SFU executam instâncias de SFU ou atuam como roteadores, dependendo do experimento. Como a avaliação se concentra na escalabilidade da rede, os clientes omitem codificação, decodificação e renderização. Cada cliente usa uma sequência de 1.800 quadros, com aproximadamente 100.000 pontos por quadro, capturada com uma câmera Intel RealSense D455 [16] com resolução de 848×484 e 30 FPS [28]. Além disso, as nuvens de pontos são codificadas com o codec Draco usando quantização de 11 bits e velocidade máxima de codificação.

4.3 Cenários de avaliação

São considerados dois cenários de conferência virtual. Para cada cenário, várias configurações são avaliadas, comparando uma linha de base com SFU centralizada com arquiteturas distribuídas usando duas SFUs (Sites 1 e 2) e quatro SFUs (uma por site). Cada configuração é testada com 8 a 64 usuários, em incrementos de 8. Os experimentos são repetidos 20 vezes e executados por 120 s, sendo os primeiros 60 s usados para a configuração da sessão e a convergência do GCC.

Cenário 1, alta escala, streaming apenas com qualidade base: Este cenário avalia a escalabilidade quando cada cliente transmite apenas uma nuvem de pontos de qualidade base (15% dos pontos, aproximadamente 12 Mbit/s). Todos os clientes recebem fluxos de todos os outros clientes, gerando um grande número de fluxos que competem por recursos de rede. A adaptação de qualidade depende das estimativas de largura de banda do GCC, usadas para selecionar aleatoriamente um subconjunto de fluxos a ser encaminhado a cada cliente. Como todos os fluxos de clientes têm a mesma taxa de bits, a largura de banda incremental necessária para suportar um usuário adicional permanece constante, o que garante um aumento mais gradual do throughput de saída.

Cenário 2, pequena escala, streaming adaptativo de alta qualidade: Este cenário modela uma situação do tipo debate ou show, em que quatro clientes (um por site) atuam como remetentes, cada um transmitindo três representações da nuvem de pontos codificadas com MDC (15%, 25% e 60% da nuvem de pontos original, a 12 Mbit/s, 18 Mbit/s e 42 Mbit/s, respectivamente), que podem ser combinadas nos receptores. Diferentemente do Cenário 1, muitos clientes receberão dados semelhantes, pois a atribuição de taxa de bits independe do número de clientes: em todas as configurações, exatamente quatro clientes transmitem três representações codificadas com MDC. No entanto, as diferenças de taxa de bits entre os níveis de qualidade não são uniformes, o que desafia o GCC a manter a latência baixa.

4.4 Métricas de avaliação

Para cada cenário de avaliação mencionado na seção anterior, as seguintes métricas são observadas e discutidas: primeiro, o throughput alcançado e a perda de pacotes. Segundo, no Cenário 1, a qualidade é determinada pelo número de representações de clientes recebidas, enquanto no Cenário 2 a qualidade é avaliada pelo número de pontos nas nuvens codificadas com MDC. Terceiro, é fornecido um detalhamento completo do sistema de ponta a ponta, com foco nos componentes de transporte. Quarto, a latência de transporte é analisada com mais detalhe, enfati-

Tabela 1: Qualidade do Cenário 1, calculada com base no número de nuvens de pontos recebidas em comparação com a quantidade máxima recebível. Ambas as configurações de SFU distribuída alcançam desempenho significativamente melhor que a SFU centralizada, com desvios-padrão altos causados por fluxos concorrentes.

#Clientes	8		16		24		32		40		48		56		64	
% da qualidade máxima	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP
SFU centralizada	100,0	0,0	100,0	0,0	43,3	29,6	23,1	21,9	14,3	17,0	11,0	15,0	8,3	7,83	6,2	11,6
2 SFUs distribuídas	100,0	0,0	100,0	0,0	90,8	18,8	78,8	31,25	30,8	31,3	21,7	21,5	14,8	17,8	9,8	13,9
4 SFUs distribuídas	100,0	0,0	100,0	0,0	91,7	7,9	75,0	24,7	32,3	33,8	21,7	22,3	15,2	16,1	10,1	14,4

Tabela 2: Qualidade do Cenário 2, calculada com base no número de pontos nas nuvens de pontos mescladas criadas a partir de representações MDC em comparação com os pontos das nuvens originais. Ambas as configurações distribuídas alcançam desempenho significativamente melhor que a SFU centralizada, com desvios-padrão altos causados por fluxos concorrentes.

#Clientes	8		16		24		32		40		48		56		64	
% da qualidade máxima	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP	Méd	DP
SFU centralizada	100,0	0,0	43,8	17,3	38,7	18,5	36,9	7,8	34,0	8,5	26,9	8,3	18,8	10,0	12,1	6,2
2 SFUs distribuídas	100,0	0,0	100,0	0,0	99,8	2,0	41,3	18,8	38,9	19,4	35,9	16,7	32,1	10,8	30,5	11,5
4 SFUs distribuídas	100,0	0,0	100,0	0,0	99,2	3,6	42,9	15,8	39,8	20,8	34,2	14,8	33,9	9,6	30,0	15,0

zando o comportamento de pior caso durante congestionamento da rede. Quinto, são examinados o uso de CPU e de memória.

4.5 Throughput e perda de pacotes

A Figura 3 mostra o throughput consumido pelos clientes, excluindo o tráfego necessário para enviar dados de um cliente à SFU ou para encaminhar conteúdo entre SFUs. As configurações distribuídas alcançam throughput significativamente maior devido ao menor compartilhamento de enlaces entre os clientes. Em contraste, a configuração centralizada alcança apenas metade do throughput para números de usuários semelhantes, pois todo o tráfego precisa atravessar a rede central.

As configurações distribuídas apresentam throughput comparável. Como a configuração com quatro SFUs encaminha dados para menos clientes, conclui-se que o throughput não é limitado pela capacidade de processamento necessária para atender um determinado número de clientes. Em vez disso, ambas as configurações distribuídas alcançam throughput semelhante principalmente por causa da topologia experimental. Na configuração com duas SFUs, os clientes dos Sites 3 e 4 estão conectados diretamente às SFUs dos Sites 1 e 2, o que permite throughput maior por enlaces não compartilhados (Figura 2). Remover esses enlaces reduziria a capacidade disponível, forçando o tráfego a compartilhar os mesmos enlaces. A configuração com quatro SFUs evita esse gargalo ao colocar cada cliente junto a uma SFU local, o que sugere que implantar SFUs mais perto dos clientes aumenta substancialmente o throughput. Combinar o throughput da Figura 3 com a topologia usada sugere que aproximadamente 50% da largura de banda de um enlace pode ser usada pelo WebRTC antes que o atraso se torne excessivo. Ambas as configurações distribuídas também apresentam um pico de throughput com 24 usuários no Cenário 2 e com 32 usuários no Cenário 1. Esse comportamento sugere a existência de um número ideal de usuários no qual a largura de banda disponível é utilizada de forma eficiente, sem interferência excessiva entre os fluxos.

Como o GCC reage preventivamente a aumentos do atraso entre pacotes, a perda de pacotes permanece baixa, com média entre 0% e 4% em todas as configurações. No entanto, como o GCC não leva em conta os aumentos simultâneos de largura de banda de múltiplos fluxos, podem ocorrer curtos períodos de perda substancial de pacotes (>20%). Esses eventos de congestionamento são breves, pois a perda de pacotes faz o GCC reduzir sua estimativa. Além disso, as retransmissões baseadas em NACK limitam o número de quadros não decodificáveis.

Figura 3: Throughput total dos dados dos clientes, com intervalos de confiança de 95%, para números de usuários de 8 a 64 (Cenário 1: apenas a menor qualidade; Cenário 2: codificação baseada em MDC; SFU centralizada, 2 SFUs distribuídas e 4 SFUs distribuídas). *(imagem omitida nesta tradução; consulte o artigo original)*

4.6 Qualidade e número de clientes recebidos

O throughput da rede impacta diretamente a qualidade por cliente. No primeiro cenário, cada cliente pode receber nuvens de pontos de todos os demais participantes; porém, como apenas uma representação de qualidade é transmitida por remetente, o cliente recebe ou uma nuvem de pontos completa ou nenhuma. A qualidade é, portanto, medida como a razão entre as nuvens de pontos recebidas com sucesso e o número máximo possível (isto é, o número total de clientes menos um), conforme reportado na Tabela 1. Com 8 e 16 usuários, todas as configurações entregam todas as nuvens de pontos a todos os clientes. À medida que o número de usuários aumenta, a configuração centralizada apresenta um declínio nas nuvens de pontos recebidas, ao passo que as SFUs distribuídas sustentam até 1,6 vez mais nuvens de pontos. Uma degradação perceptível nas configurações distribuídas ocorre apenas a partir de 40 usuários, enquanto na configuração centralizada ela aparece com 24 usuários.

O Cenário 2 emprega codificação baseada em MDC, que permite aos clientes receber múltiplas representações de qualidade de uma nuvem de pontos. Como cada representação contribui com pontos distintos, a qualidade é definida como a razão entre os pontos da nuvem mesclada e os da original. A pontuação final de qualidade é calculada como a média de todas as nuvens de pontos recebidas, com as nuvens ausentes contribuindo com 0%. Como mostra a Tabela 2, ambas as configurações distribuídas alcançam qualidade substancialmente maior que a SFU centralizada. O desvio-padrão relativamente grande se deve principalmente ao conjunto limitado de sete combinações MDC distintas, que diferem em pelo menos 10% dos pontos.

4.7 Detalhamento da latência de ponta a ponta

A Figura 4 mostra a latência média de ponta a ponta e a qualidade para um dos quatro nós remetentes no Cenário 2 com 32 participantes. A configuração centralizada alcança latência ligeiramente menor devido à menor sobrecarga de decodificação, resultante do menor throughput discutido na Seção 4.5, que limita a entrega a representações de menor qualidade. Focando nos componentes relacionados ao transporte (Cliente–SFU, SFU–SFU e SFU–Cliente), todas as configurações diferem em apenas alguns milissegundos. Esses componentes são medidos como o tempo entre o envio do primeiro pacote e o recebimento do último. Observa-se uma pequena diferença entre as configurações de 2 e 4 SFUs: a maior latência cliente–SFU no caso de 2 SFUs resulta do roteamento, pois os clientes dos Sites 3 e 4 precisam transmitir dados a SFUs localizadas em outros sites. Consequentemente, o tráfego pode atravessar vários saltos, cada um adicionando atraso (Figura 2). Esse efeito não ocorre na configuração centralizada, em que todo o tráfego segue um caminho pelo nó central. Ainda assim, quando remetente e receptor estão no mesmo local, ambas as configurações distribuídas alcançam latência menor que a centralizada, por evitarem totalmente os enlaces entre sites.

Figura 4: Latência média de ponta a ponta por remetente em um cenário baseado em MDC com 32 clientes e quatro remetentes (SFU centralizada: qualidade 36%; 2 SFUs distribuídas: 42%; 4 SFUs distribuídas: 41%). A latência WebRTC é dividida em 3 categorias (Cliente–SFU, SFU–SFU, SFU–Cliente), além de captura, pré-processamento, decodificação e renderização.
(imagem omitida nesta tradução; consulte o artigo original)

No entanto, a configuração com 4 SFUs encaminha mais dados (210 Mbit/s quando o fluxo de um cliente é encaminhado para três SFUs, contra 140 Mbit/s quando dois clientes são encaminhados para uma única SFU), aumentando ligeiramente a latência de rede induzida pelo encaminhamento. Esse compromisso ressalta a importância tanto do número quanto do posicionamento das SFUs e sugere que uma orquestração dinâmica poderia adaptar as implantações com base nas condições atuais da rede e nas localizações dos clientes.

A câmera introduz a maior latência, por causa do atraso entre a captura da imagem e o processamento. Em razão da codificação baseada em MDC, a latência de pré-processamento permanece constante entre as configurações e independe do número de clientes. Em contraste, a latência de decodificação aumenta com o número de descrições recebidas dos outros remetentes.

4.8 Latência de transporte de pico

Grandes picos de latência podem diminuir severamente a imersão, e até atrasos breves e inconsistentes podem atrapalhar a interação. A Figura 5 apresenta a latência de rede mediana junto aos valores dos percentis 90 e 99, que, em conjunto, indicam tanto a frequência quanto a severidade dos picos de latência.

Como mostra a Figura 5, a latência mediana permanece estável, aumentando apenas ligeiramente com o número de usuários. A configuração centralizada apresenta a menor latência quando comparada à configuração de 2 SFUs. No entanto, com quatro SFUs, a latência se torna comparável à do caso centralizado, graças ao melhor roteamento quando remetentes e receptores estão no mesmo local. De acordo com os resultados da Seção 4.5, o GCC contribui para uma latência estável ao reduzir agressivamente o throughput diante de congestionamento. Mesmo assim, grandes picos ainda podem ocorrer quando vários fluxos aumentam suas taxas simultaneamente. Os resultados do percentil 99 mostram que, devido ao uso mais eficiente da largura de banda, as configurações distribuídas apresentam um aumento mais suave da latência de pico à medida que o número de usuários cresce. Na configuração centralizada, os picos de latência surgem entre 24 (Cenário 1) e 32 (Cenário 2) usuários, ao passo que as configurações distribuídas só apresentam valores semelhantes com 40 usuários. De forma consistente com a mediana, o percentil 90 permanece estável e baixo o suficiente para facilitar a interação dos usuários em todas as configurações.

Figura 5: A latência mediana é estável para números de usuários de 8 a 64. As latências de pico, representadas pelos percentis, aumentam mais, sendo o percentil 90 estável.
(imagem omitida nesta tradução; consulte o artigo original)

4.9 Recursos do sistema

A Figura 6 mostra que o uso de memória em ambos os cenários é substancialmente maior para a SFU centralizada do que para as duas configurações distribuídas. Embora cada SFU distribuída individualmente consuma menos memória, o uso agregado entre todas as SFUs distribuídas é comparável ao de uma SFU centralizada, ou ligeiramente menor. No Cenário 1, o alto uso é atribuído às retransmissões baseadas em NACK. Com o NACK habilitado, todo pacote enviado a pelo menos um cliente precisa ser temporariamente armazenado em buffer. Na configuração centralizada, atender a todos os clientes e encaminhar muitas *tracks* distintas aumenta o número de buffers, fazendo o uso de memória crescer rapidamente com o número de usuários. Em contraste, quando os clientes recebem dados idênticos, como no Cenário 2, os buffers podem ser compartilhados, mantendo o uso de memória baixo. Além disso, reduzir o tamanho dos buffers é impraticável, pois a alta taxa de bits das nuvens de pontos faria os buffers transbordarem rapidamente. A Figura 7 mostra que o uso de CPU aumenta apenas modestamente à medida que o número de usuários cresce. Uma exceção ocorre no Cenário 2: com oito usuários, o uso de CPU é significativamente maior que no Cenário 1, refletindo a maior taxa de bits inicial de um único objeto codificado com MDC (70 Mbit/s contra 12 Mbit/s). Como discutido na Seção 4.5, o GCC estabiliza em seguida o throughput e a latência, fazendo o uso de CPU convergir quando o número de usuários ultrapassa 32. Os picos de CPU observados com 24 e 32 usuários coincidem com os picos de throughput da Figura 3.

Figura 6: Uso de RAM por SFU, com intervalos de confiança de 95%, para números de usuários de 8 a 64.
(imagem omitida nesta tradução; consulte o artigo original)

Figura 7: Uso médio de CPU em todos os núcleos, com intervalos de confiança de 95%, para números de usuários de 8 a 64.
(imagem omitida nesta tradução; consulte o artigo original)

5 CONCLUSÃO E TRABALHOS FUTUROS

Este artigo apresentou uma arquitetura escalável de streaming de vídeo volumétrico muitos-para-muitos que utiliza unidades de encaminhamento seletivo (SFUs) distribuídas para WebRTC, a fim de melhorar o desempenho, aumentar o número de usuá-

rios suportados e reduzir a utilização de recursos. Avaliamos configurações centralizadas e distribuídas, com duas e quatro SFUs, para números de usuários de 8 a 64 em dois cenários: streaming de nuvens de pontos de baixa qualidade em alta escala e transmissão multiqualidade seletiva baseada em codificação por múltiplas descrições (MDC) com remetentes limitados. Os resultados mostram que as configurações distribuídas utilizam a largura de banda de forma mais eficiente, dobrando o throughput ao reduzir os enlaces compartilhados. Isso permite maior cobertura de clientes no primeiro cenário e qualidade substancialmente maior no segundo, com até 147% mais pontos recebidos com 64 usuários. Embora o Google Congestion Control (GCC) mantenha baixa perda de pacotes e latência estável em todas as configurações, a latência de pior caso difere acentuadamente: a configuração centralizada ultrapassa 100 ms com 32 usuários, enquanto as distribuídas só alcançam esse limite acima de 48 usuários. Trabalhos futuros estenderão a arquitetura para uma topologia totalmente hierárquica, permitindo encaminhamento em múltiplos níveis por regiões mais amplas e suportando mais de 64 usuários. Também serão exploradas a escalabilidade dinâmica de SFUs com transferência (*handover*) contínua de clientes e a atribuição adaptativa de clientes a SFUs com base em métricas como latência e o campo de visão (FoV) atual de todos os usuários.

AGRADECIMENTOS

Este trabalho foi financiado pela União Europeia (projeto SPIRIT, Acordo de Subvenção 101070672, https://www.spiritproject.eu/).

REFERÊNCIAS (NO IDIOMA ORIGINAL)

[1] A. O. Al-Abbasi, V. Aggarwal, and M.-R. Ra. 2019. Multi-tier caching analysis in cdn-based over-the-top video streaming systems. IEEE/ACM Transactions on Networking, 27, 2, 835–847.
[2] E. André, N. Le Breton, A. Lemesle, L. Roux, and A. Gouaillard. 2018. Comparative study of webrtc open source sfus for video conferencing. In 2018 Principles, Systems and Applications of IP Telecommunications (IPTComm). IEEE, 1–8.
[3] A. Bentaleb, M. Lim, S. Hammoudi, S. Harous, and R. Zimmermann. 2025. Solutions, challenges, and opportunities in volumetric video streaming: an architectural perspective. ACM Transactions on Multimedia Computing, Communications and Applications, 21, 7, 1–35.
[4] A. Bentaleb, Z. Zhan, F. Tashtarian, M. Lim, S. Harous, C. Timmerer, H. Hellwagner, and R. Zimmermann. 2022. Low latency live streaming implementation in dash and hls. In Proceedings of the 30th ACM International Conference on Multimedia, 7343–7346.
[5] B. Bross, Y.-K. Wang, Y. Ye, S. Liu, J. Chen, G. J. Sullivan, and J.-R. Ohm. 2021. Overview of the versatile video coding (vvc) standard and its applications. IEEE Transactions on Circuits and Systems for Video Technology, 31, 10, 3736–3764.
[6] G. Carlucci, L. De Cicco, S. Holmer, and S. Mascolo. 2016. Analysis and design of the google congestion control for web real-time communication (WebRTC). In Proceedings of the 7th International Conference on Multimedia Systems, 1–12.
[7] I. Chicano-Capelo, F. Gortázar, and M. Gallego. 2025. Quality of experience under huge load for webrtc applications: a case study of three media servers. IEEE Access.
[8] J. Choi, J.-B. Jeong, S. Lee, and E.-S. Ryu. 2022. Overview of the volumetric video capturing system for immersive media. In 2022 13th International Conference on Information and Communication Technology Convergence (ICTC). IEEE, 574–577.
[9] M. Dasari, E. Lu, M. W. Farb, N. Pereira, I. Liang, and A. Rowe. 2023. Scaling vr video conferencing. In 2023 IEEE Conference Virtual Reality and 3D User Interfaces (VR). IEEE, 648–657.
[10] M. De Fré, J. van der Hooft, T. Wauters, and F. De Turck. 2024. Scalable mdc-based volumetric video delivery for real-time one-to-many webrtc conferencing. In Proceedings of the 15th ACM multimedia systems conference, 121–131.
[11] R. Deshmukh, N. Nand, A. Pawar, D. Wagh, and A. Kudale. 2023. Video conferencing using webrtc. In 2023 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS). IEEE, 857–864.
[12] Google. 2023. Draco. (2023). https://google.github.io/draco/.
[13] D. Dziwis and H. von Coler. 2023. The entanglement: volumetric music performances in a virtual metaverse environment. Journal of Network Music and Arts, 5, 1, 3.
[14] M. Hosseini and C. Timmerer. 2018. Dynamic adaptive point cloud streaming. In Proceedings of the 23rd Packet Video Workshop, 25–30.
[15] 2025. Imec virtual wall testbed. (2025). https://doc.ilabt.imec.be/ilabt/virtualwall/.
[16] [SW], Intel Realsense 2025. url: https://www.intelrealsense.com/.
[17] Y. Jin, K. Hu, J. Liu, F. Wang, and X. Liu. 2023. From capture to display: a survey on volumetric video. arXiv preprint arXiv:2309.05658.
[18] R. A. Kırmızıoğlu, A. M. Tekalp, and B. Görkemli. 2025. Distributed virtual selective-forwarding units and sdn-assisted edge computing for optimization of multi-party webrtc videoconferencing. Signal Processing: Image Communication, 130, 117173.
[19] S. F. Langa, M. Montagud, G. Cernigliaro, and D. R. Rivera. 2022. Multiparty holomeetings: toward a new era of low-cost volumetric holographic meetings in virtual reality. Ieee Access, 10, 81856–81876.
[20] L.-H. Lee, T. Braud, P. Y. Zhou, L. Wang, D. Xu, Z. Lin, A. Kumar, C. Bermejo, P. Hui, et al. 2024. All one needs to know about metaverse: a complete survey on technological singularity, virtual ecosystem, and research agenda. Foundations and trends® in human-computer interaction, 18, 2–3, 100–337.
[21] G. Li, W. Gao, and W. Gao. 2024. Mpeg geometry-based point cloud compression (g-pcc) standard. In Point Cloud Compression: Technologies and Standardization. Springer, 135–165.
[22] S. Maheshwari, D. Raychaudhuri, I. Seskar, and F. Bronzino. 2018. Scalability and performance evaluation of edge cloud systems for latency constrained applications. In 2018 IEEE/ACM Symposium on Edge Computing (SEC). IEEE, 286–299.
[23] S. A. Mahmood and M. Kherallah. 2023. Evaluation of the existing web real-time signaling mechanism for peer-to-peer communication: survey. In International Conference on Advanced Engineering, Technology and Applications. Springer, 298–310.
[24] P. Malinovskiy. 2025. Solving the problem of poor internet connectivity in dhaka: innovative solutions using advanced webrtc and adaptive streaming technologies. arXiv preprint arXiv:2506.17343.
[25] 2025. Mdc-based webrtc point cloud streaming. (2025). https://github.com/idlab-discover/pc-webrtc-m2m.
[26] M. Medvetskyi, M. Beshley, I. Scherm, O. Urikova, H. Beshley, and T. Dierkes. 2024. Sdn-based 6g video content delivery technique via multi-path routing, multi-server architecture and multi-connectivity. In 2024 IEEE 17th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET). IEEE, 169–173.
[27] P. Nuño, F. G. Bulnes, J. C. Granda, F. J. Suárez, and D. F. García. 2018. A scalable webrtc platform based on open technologies. In 2018 International Conference on Computer, Information and Telecommunication Systems (CITS). IEEE, 1–5.
[28] 2025. Point cloud sequence used in experiments. (2025). https://users.ugent.be/~madfr/pc-streaming-mmsys2024/frames.zip.
[29] M. Rudolph et al. 2024. Transcoding v-pcc point cloud streams in realtime. ACM Transactions on Multimedia Computing, Communications and Applications.
[30] S. Subramanyam, I. Viola, A. Hanjalic, and P. Cesar. 2020. User centered adaptive streaming of dynamic point clouds with low complexity tiling. In Proceedings of the 28th ACM international conference on multimedia, 3669–3677.
[31] [SW], Unity 2023. url: https://unity.com.
[32] N. Van der Meer, V. van der Werf, W.-P. Brinkman, and M. Specht. 2023. Virtual reality and collaborative learning: a systematic literature review. Frontiers in Virtual Reality, 4, 1159905.
[33] J. van der Hooft, H. Amirpour, M. T. Vega, Y. Sanchez, R. Schatz, T. Schierl, and C. Timmerer. 2023. A tutorial on immersive video delivery: from omnidirectional video to holography. IEEE Communications Surveys & Tutorials, 25, 2, 1336–1375.
[34] J. van der Hooft, T. Wauters, F. De Turck, C. Timmerer, and H. Hellwagner. 2019. Towards 6dof http adaptive streaming through point cloud compression. In Proceedings of the 27th ACM International Conference on Multimedia, 2405–2413.
[35] I. Viola, J. Jansen, S. Subramanyam, I. Reimat, and P. Cesar. 2023. Vr2gather: a collaborative social vr system for adaptive multi-party real-time communication. IEEE MultiMedia, 30, 2, 48–59.
[36] Y. Wang, A. R. Reibman, and S. Lin. 2005. Multiple description coding for video delivery. Proceedings of the IEEE, 93, 1, 57–70.

[37] S. M. Yamijala, N. Nimbrain, and R. Bansal. 2025. Enhancing remote workspaces: the role of virtual reality in shaping the virtual work environment. Optimizing virtual reality and metaverse for remote work and virtual team collaboration, 171–190.
[38] D. Yuan et al. 2025. Understanding operational cdn live streaming: a measurement study on performance, costs and enhancements. IEEE Transactions on Circuits and Systems for Video Technology.